czii-cryo-et-object-identification

10位解法

Rank	10
Team	Josef Slavicek
Author	Josef Slavicek
Topic ID	561844
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561844
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

まず主催者にお詫びしたいと思います。主催者は、CryoET の処理に大きな進歩をもたらすことを願って、とてもよく準備された興味深いコンペを提供してくれました。それなのに、私から届けられたのは、昔ながらの 3DUNet のアンサンブルにすぎません。

また、役に立つ notebook の作者の皆さん、特にこれの作者に感謝したいと思います。それらがなければ、動くものを何も作れなかったでしょう。

簡単な説明

私の解法は 9 個の 3DUnet のアンサンブルです。1つを除くすべてのモデルは、まずシミュレーションデータで事前学習し、その後コンペの学習データでファインチューニングしました。アンサンブルは、等しい重みの logit を単純に平均するだけです。その後、確率を計算し、閾値をかけて検出領域を得て、領域を個々の粒子に分割する後処理を適用します。私の学習はすべて、実際には Optuna によるハイパーパラメータ探索のセッションで、その中から、取っておいたローカルの検証セットに基づいて最良のモデルを選びました。

検証

学習 / 検証の分割は、単に experiment ID で行いました。コンペの学習データでのファインチューニングでは、6つのボリュームで学習し、残りの1つで検証します。シミュレーションデータでの事前学習でも、検証には学習データの experiment の1つでの性能を使っています。

データ処理とデータ拡張

使ったデータ処理は、monai.transforms.NormalizeIntensityd による正規化だけです。データ拡張については、monai.transforms の RandFlipd、RandRotated、RandRotate90d を使っています。学習中には mixup の単純な実装も使っていますが、Optuna はそれにとても小さな alpha（約 0.03）を選びました。

NN のアーキテクチャ

私のネットはすべて同じアーキテクチャです: monai.networks.net.UNet、spatial_dims=3、channels=(48, 64, 80, 80, 128)、stride_patterns=(2,2,2,1)。

学習

私の学習はすべて Optuna のハイパーパラメータ探索のセッションとして行い、内側の学習は pytorch lightning で行いました。AdamW optimizer と cosine LR scheduler を使っていました。Optuna は Adam の beta パラメータに、いくつか珍しい値（例: beta1=0.7、beta2=0.9996）を選びました。

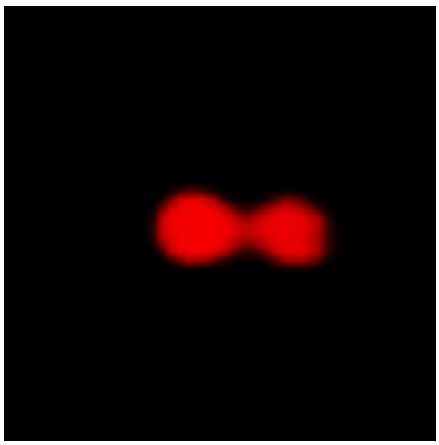
学習の目的関数

コンペのデータでのファインチューニングでは、UNet に粒子クラスのマップを予測させていました。loss 関数には、Tversky loss と多クラスの crossentropy の重み付きの組み合わせを使っていました。Tversky loss への入力の logit はクリップしました。シミュレーションデータでの事前学習では、3種類の目的関数を使いました。

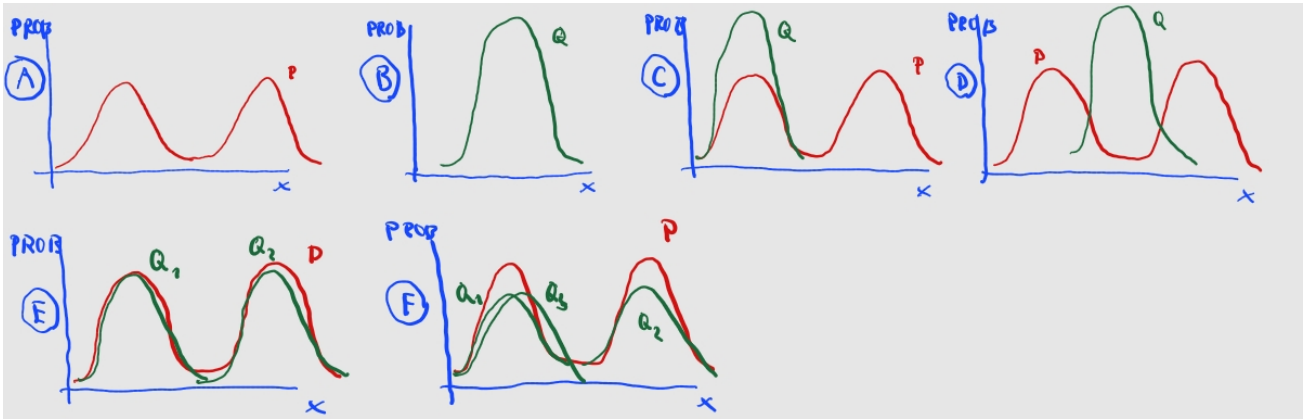
1. ファインチューニングと同じもの。
2. 粒子の向きのベクトルを追加の目的変数にしたもの。これでネットワークが粒子の形を理解するよう強制されることを期待しましたが、実際には、ネットワークはランダムな推測より良く向きを予測できませんでした。
3. 1. の初期の実装で、向きが本当にランダムになってしまうバグがあったもの。皮肉なことに、これが最も成功した事前学習で、最終的な9つのネットワークのうち4つに含まれています

後処理

融合し、確率を計算し、閾値をかけた後には、粒子を表す二値マップが得られます。ほとんどの場合、正の値の孤立したクラスタは実際に単一の粒子に対応しています。しかし、ときどき次のようなものが見られました。



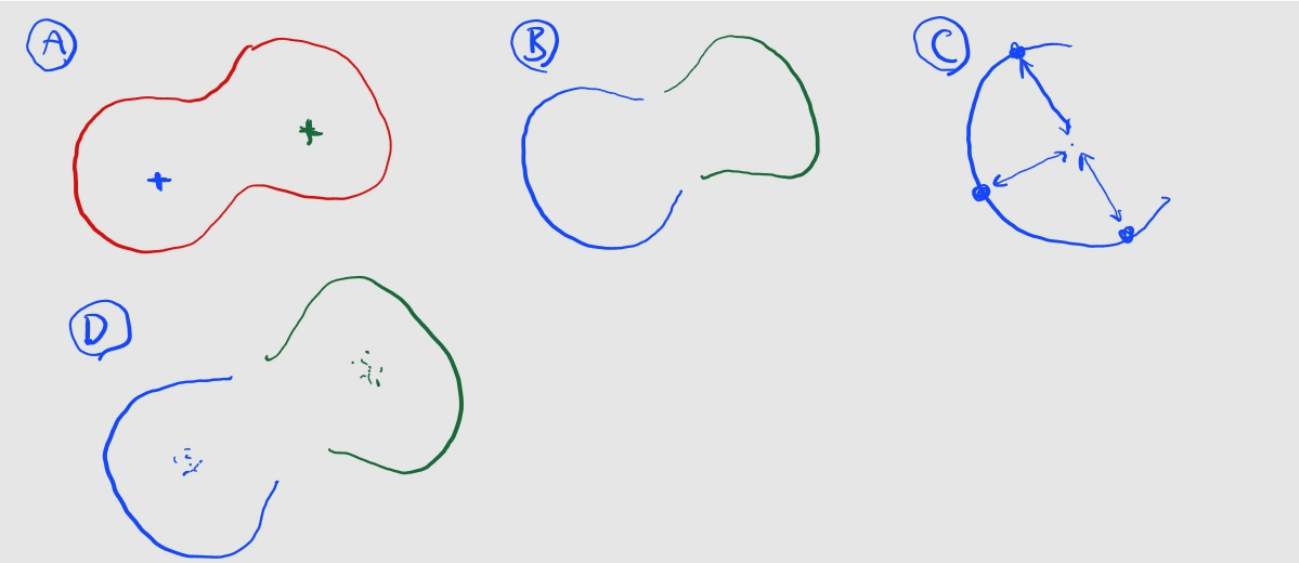
そこで、このような多重検出を認識して分割する必要があります。おそらく、これを行うための確立された実績ある解決法があるのだと思いますが、私はこれを車輪の再発明をする良い機会と捉え、最終的に次のようにしました（分かりやすさのため 1D 画像で説明します）。



モデルの予測である確率密度関数（pdf）があります（画像 A）。これを P と呼びましょう。単一の粒子の pdf（Q と呼びましょう）がどのような形かはよく分かっています（画像 B）。ただ、それがどこにあるのか、いくつあるのかが分からないだけです。しかし、KL ダイバージェンスを最小化するように Q を P の上に置けば、まさに望む

ものが得られます（画像 C）。ただし、KL は対称ではないので注意が必要です。P と Q を KL の式に逆の順序で入れると、望ましくない結果になります（画像 D）。しかし正しく行えば、最初の粒子を置いたことになり、別の粒子を置くと、それは P の別の山を占めます（画像 E）。これを続けることができます。置いた Q 同士が重なり始めたら（画像 F）、終わりです。

これが、私のクレイジーな KL の仕組みの前半です。残りを説明するために、2D の表示に切り替え、前の画像の色の意味は忘れてください。今度は違う意味になります。



モデルの予測があり、そこから二値マスクを描け、そこから境界を検出できます。画像 A の赤い線です。そして、個々の粒子の pdf の重心（緑と青の十字）が分かっています。これらの重心を使うだけでも良い予測になりますが、次の小技で少し改善しました。赤い境界を、個々の粒子に属する部分に分けることができます（画像 B）。そして、それらを別々に処理します。ランダムに4点を選び（実際の 3D の問題では4点、2D の画像では3点になります）、その外心（それらから等距離にある唯一の点）を求めると、粒子の重心の良い候補が得られます（画像 C）。これを使って粒子中心の候補の点群を得て、このクラスタの密な中心部分を平均することで、最終的な重心の予測を求めます（画像 D）。

以上です。すみません。我慢できませんでした :-)

この KL マジック全体で、LB スコアが約 0.015 改善し、P100 で実行に 90 分かかります。

補足Q&A

Reza Paraan — 質問

こんにちは Josef。参加と貴重なレポートに改めて感謝します。後処理について質問があります。このステップの後に粒子の直径を計算していますか。もしそうなら、その情報を使って、その粒子が正しいクラスに属するかどうかを判断していますか。たとえば、多くの apoferritin 粒子は互いに集まってクラスタになります。あなたの後処理がなければ、このクラスタは VLP のような1つの大きな粒子に見えるかもしれません。あなたの方法がそのような粒子クラスの混同に対処しているのか、興味があります。どうもありがとうございます。

Josef Slavicek — 回答

こんにちは Reza。返信が遅くなってすみません。この discussion を見ていませんでした。

いいえ、各クラスの予測は別々に処理しています。粒子の直径はこの計算に固定パラメータとして入っていて、私の後処理がしようとしているのはただ一つのこと、つまり、そのクラスの NN 出力の上に、単一粒子の予測を配置することだけです。